ChatGPT

开源项目《QQ群搬史(屎)分析仪》深度调研与方案报告

执行摘要

本次调研以“QQ群聊天记录导入 + 多维分析（含表情包/图片）+ RAG/Embeddings/LLM”为目标，系统检索了当前公开可见的相关工具与项目，结论是：**“QQ群搬史分析仪”这一“全链路（导出/采集 → 结构化存储 → 多模态理解 → 检索与RAG → 可视化与报告）一体化”形态在开源生态中尚未出现成熟、通用、可复用且合规风险可控的完整对标产品**；但已经存在若干“强相关组件”覆盖了链路的不同段落：
一是 **QCE/qq-chat-exporter** 解决“导出/备份/下载媒体”问题，且支持 HTML/JSON/ZIP 导出、定时备份、批量导出与表情包导出，并能完整下载图片/视频；其 WebUI 使用 Token 与 IP 白名单等机制实现本地访问控制。[1]
二是 **ChatLab** 提供“本地优先、流式解析、多线程 worker、SQL+AI Agent”的“分析/探索”形态，并明确支持 QQ 导入与统一化数据模型。[2]
三是 **QQChatAnalyzer / QQ-Chat-AI-Analyzer / QQ Summary Genius** 等项目分别覆盖“文本统计+社交网络可视化”“年度报告式生成”“文本清洗+调用第三方大模型总结”等子场景，但往往**缺少图片OCR/表情包语义、缺少向量检索与RAG、缺少可扩展插件体系与系统级隐私合规设计**。[3]

因此，若你要开发一个真正“能用、可扩展、开源、非商业化（运营层面）”的《QQ群搬史(屎)分析仪》，推荐采用“**本地优先 + 导入多源数据（优先文件导入）+ 可选官方机器人实时增量**”的合规路线：
- **历史记录**首选“用户主动导出的文件”导入（QCE JSON/ZIP/HTML、QQ 消息管理器 TXT 等），避免“未授权抓取/逆向登录/读取他人数据库”的高风险路线；QCE 已明确支持导出网页/数据格式并下载图片视频，适合作为主要数据来源。[4]
- **实时增量**若确需自动化，可优先使用 **QQ 官方机器人平台**（API + webhook 事件），其官方文档宣称覆盖“频道、群、单聊”等场景，并在运营规范中强调“采集用户数据需经用户同意、披露用途并按请求删除”等要求；但官方机器人通常只覆盖“新消息流”，难以补齐历史消息。[5]
- 多模态能力建议用 **PaddleOCR**（中文 OCR 生态成熟、文档齐全）+ **CLIP**（图文共同嵌入）+ **Qwen2.5-VL**（多模态理解/图片解释）组合；RAG 采用“Hybrid retrieval（稠密+稀疏）+ Reranker”路径，并参考 RAG 原始论文“检索增强生成”范式降低幻觉。[6]
- 向量数据库方面：小规模/本地优先可先用 **Chroma**，规模扩大再上 **Milvus/Weaviate**；若需要托管与合规认证则考虑 **Pinecone**，但其为托管商业服务且成本/数据出境需评估。[7]

最后，本项目天然具有“搬史/曝光/网暴”滥用风险，你需要在产品与开源治理中把“合规与反滥用”作为一等公民：默认本地处理、敏感字段脱敏、权限与审计、导出水印、明确用户必须获得群成员同意等；这与《个人信息保护法》强调的“合法、正当、必要、最小范围、透明、保障安全”等原则方向一致。[8]

市场与竞品检索结果

本节按“**与QQ群聊天记录分析最相关**”的标准列出当前已发现项目（开源为主，含部分闭源/官方平台）。为满足你的目标字段，我将信息拆为两张表：基础信息表 + 能力矩阵表。

基础信息对比表

项目	仓库/官网（链接用代码展示）	主要定位	技术栈/形态（从公开信息归纳）	许可	活跃度（可量化信息）
QCE / qq-chat-exporter	`https://github.com/shuakami/qq-chat-exporter`；`https://qce.sdjz.wiki/`	QQ 聊天记录导出/备份（含媒体、表情包）	本地运行 + WebUI；支持 Shell(无头)/Framework(插件)两种模式；导出 HTML/JSON/ZIP；定时/批量导出；表情包导出；可下载图片/视频。[9]	GPL-3.0 [10]	GitHub 显示约 2.5k stars、369 commits。[10]
napcat-qce-python	`https://github.com/streetartist/napcat-qce-python`	QCE API 的 Python SDK（自动化导出/监听）	Python 包，封装 QCE API、自动令牌获取、WebSocket 事件、定时与批量导出、表情包导出等。[11]	GPL-3.0 [11]	GitHub 显示 15 stars、7 commits。[11]
ChatLab	`https://github.com/hellodigua/ChatLab`；`https://chatlab.fun`	本地优先的多平台聊天记录分析（SQL+AI Agent）	Electron 桌面端；流式解析+多 worker；统一化数据模型；支持 QQ/微信/WhatsApp 等；提供系统架构说明：主进程+Worker 层+Renderer 层（Vue3/Nuxt UI/Tailwind）。[2]	AGPL-3.0 [2]	GitHub 显示约 4.8k stars、369 commits。[2]

项目	仓库/官网（链接用代码展示）	主要定位	技术栈/形态（从公开信息归纳）	许可	活跃度（可量化信息）
QQChatAnalyzer	`https://github.com/CutrelyAlex/QQChatAnalyzer`	QQ 文本聊天记录分析平台（统计/关系网络/AI摘要）	Python Web 应用；导入 QCE JSON 或 QQ 导出的 TXT；支持个人/群聊分析、社交网络可视化、AI 摘要；明确“不支持图片分析”。[12]	MIT [13]	GitHub 显示约 40 stars、59 commits。[13]
QQ-Chat-AI-Analyzer	`https://github.com/JUSTMONIKA2022/QQ-Chat-AI-Analyzer`	基于 LLM 的群聊总结/年度报告生成	本地部署；仅适配 QCE 导出的 JSON；作者提示大文件会影响效果、建议≤120MB；并明确提示可能将聊天记录完整上传至模型服务商。[14]	GPL-3.0 [14]	GitHub 显示约 21 stars。[14]
annual-report-builder	`https://github.com/ping-xiong/annual-report-builder`；`https://gitee.com/LiangJiaping/annual-report-builder`	QQ/微信聊天记录年度报告（模板+词云+导出）	Electron 桌面端；支持年度/月度/自定义时间范围；群聊/私聊；词云/分词设置；本地运行；可导出网页/长图/纯文本；提示“安装路径不能有中文否则闪退”。[15]	MIT [16]	GitHub 显示 26 stars；Releases 最新为 0.3.2（2022-02-09）。[16]
QQLog	`https://github.com/usamimeri/QQLog`	TXT 聊天记录读取+统计+可视化（含情感/图片频次）	Python 脚本/库形态；从 TXT 提取到 JSON/DataFrame，做关键词、时间序列、图片/表情频次、情感分析与词云可视化。[17]	未在页面摘要中看到明确许可证声明（需进一步核验）[17]	GitHub 显示 9 stars、22 commits。[17]

项目	仓库/官网（链接用代码展示）	主要定位	技术栈/形态（从公开信息归纳）	许可	活跃度（可量化信息）
QQ Summary Genius	`https://github.com/JyiDeng/QQ-Summary-Genius`	QQ 聊天 TXT 清洗 + 调用多家大模型 API 总结	Python 脚本；支持按日期过滤、移除 QQ 号/昵称/系统消息/重复消息、移除[图片]标记；支持 SiliconFlow/OpenAI/Anthropic 等 API；输出 Markdown 总结。[18]	MIT [18]	GitHub 显示 ~33+ stars（页面摘要有浮动）。[19]
chatlog（已移除代码）	`https://github.com/sjzar/chatlog`	曾尝试做“聊天记录工具”，现仅剩移除声明	作者公告称收到微信官方函件，认为核心功能存在合规风险，因此移除全部代码与提交历史，仅保留说明。[20]	不适用（仓库仅公告）[20]	该历史事件对“聊天记录类工具”的合规风险评估具有强警示意义。[20]
QQ 官方机器人平台（闭源/官方）	`https://bot.q.qq.com/wiki/`	官方 API/webhook 的群/频道/单聊机器人接入	官方开放平台；提供 API 与 webhook 事件；官方介绍宣称打通频道、群、单聊等场景。[21]	平台协议约束（非开源）[22]	官方平台持续运营（以官方文档为准）。[21]

关键能力矩阵对比表

说明：这里的“支持”以项目 README/文档明确描述为准；未明确提到的不做强行推断。

项目	QQ 聊天记录导入/采集方式	是否支持表情包/图片“导入或导出”	是否支持图片语义分析（OCR/特征/Caption）	是否有 Embeddings/向量检索	是否有 RAG	是否有 LLM 集成
QCE / qq-chat-exporter	扫码登录后导出；支持 HTML/JSON/ZIP；也支持无 Token 的“独立模式”查看旧备份。[23]	支持完整下载图片/视频；支持表情包导出。[24]	文档未宣称“OCR/视觉理解”，偏导出工具。[25]	否	否	否
napcat-qce-python	通过 API 调度 QCE 导出/监听；可自动获取 Token。[11]	提供表情包导出、消息导出等封装。[11]	否	否	否	否
ChatLab	导入聊天导出文件；明确支持 QQ 等多平台；并做“跨平台标准化”。[2]	文档强调“聊天导出”统一模型，但未在摘要中逐项列出图片处理细节（需进一步读其导入规范）。[2]	具备 AI Agent + Tool Calling 工作流（可用于总结/分析/搜索），但是否含 OCR/图像嵌入需进一步查规范。[2]	未在摘要中明确说明向量库/embedding（不强推断）[2]	未明确（但“AI+检索/分析”能力较强）[2]	是（AI Agents + Function Calling）。[2]

项目	QQ 聊天记录导入/采集方式	是否支持表情包/图片“导入或导出”	是否支持图片语义分析（OCR/特征/Caption）	是否有 Embeddings/向量检索	是否有 RAG	是否有 LLM 集成
QQChatAnalyzer	导入 QCE JSON 或 QQ 消息管理器 TXT。[13]	明确“不支持图片分析”；TXT 导入会损失表情/文件等信息量。[13]	否（明确不支持）。[13]	否	否	是（可配置第三方 AI 服务商）。[26]
QQ-Chat-AI-Analyzer	仅支持 QCE 导出的 JSON。[14]	未宣称图片/表情分析（核心是报告生成）。[14]	未宣称	否	否	是（依赖外部 LLM，且提示可能上传全部聊天记录）。[14]
annual-report-builder	导入 QQ/微信导出的 TXT。[27]	TXT 路径通常仅记录“[图片]/[表情]”标记，非媒体语义分析（项目定位为模板报告）。[28]	否	否	否	否
QQLog	TXT→JSON/DataFrame；含“发图和表情包统计”等。[17]	统计层面支持（频次），并非图像内容理解。[17]	否（未宣称 OCR/视觉嵌入）。[17]	否	否	未宣称

项目	QQ 聊天记录导入/采集方式	是否支持表情包/图片“导入或导出”	是否支持图片语义分析（OCR/特征/Caption）	是否有 Embeddings/向量检索	是否有 RAG	是否有 LLM 集成
QQ Summary Genius	TXT 输入；做清洗后调用多家 API 总结。[18]	清洗步骤会移除 [图片] 标记等。[18]	否	否	否	是（SiliconFlow/OpenAI/Anthropic 等）。[29]

小结：
- “可用的 QQ 记录导出/备份”已相对成熟（QCE），并且具备媒体下载与表情包导出能力。[24]
- “分析端”分为两派：桌面本地优先的 ChatLab（更系统）与脚本/小应用（更轻，但能力碎片化）。[30]
- **缺口主要集中在：多模态（表情包/图片的OCR+语义）、向量检索/嵌入、RAG 问答、可扩展的插件与合规治理框架。**（这是你项目的机会点。）

代表性项目技术解剖

本节选取 5 个“代表性项目/组件”做深入技术分析：QCE（导出/采集）、ChatLab（本地分析平台）、QQChatAnalyzer（Web分析+AI摘要）、QQ-Chat-AI-Analyzer（年度报告式 LLM 生成）、NapCat（QQ接入底座但许可特殊）。选择理由是它们分别代表链路的关键位置与工程权衡点。

QCE / qq-chat-exporter 深入分析

定位与边界
QCE 的目标是把 QQ 聊天记录保存到电脑，支持导出网页（HTML）/数据（JSON）等，并能完整下载图片与视频；同时支持表情包导出、定时备份、批量导出。[31] 这使它成为“搬史分析仪”的最佳上游数据源之一：你不必自己实现 QQ 数据抽取，只需对接其导出物或 API。

运行/部署模式
QCE 文档明确两种模式：
- **Shell 模式**：独立无头 QQ，适合服务器/自动化备份；QQ 在后台无头运行。[23]
- **Framework 模式**：作为 QQNT 插件运行，共享登录状态，适合 Windows 桌面用户。[23]

其 WebUI 常见入口为 `http://localhost:40653/qce-v4-tool`，并通过 `security.json` 中的 `accessToken` 登录；同时存在 `allowedIPs` 与 `tokenExpired` 字段，体现了**本地服务鉴权 + IP 白名单**的安全思路。[32]

高层架构与数据流（推断以公开文档为依据）

```
flowchart TB
  subgraph User[用户设备/服务器]
    QQ[QQ / NTQQ 登录态]
    QCE[QCE 运行时\n(Shell或Framework)]
    UI[WebUI: /qce-v4-tool\n端口40653]
    SEC[security.json\naccessToken + allowedIPs]
```

```
    OUT[导出文件\nHTML/JSON/ZIP + 媒体(图片/视频) + 表情包]
  end

  QQ --> QCE
  QCE --> OUT
  SEC --> UI
  UI --> QCE

  classDef local fill:#f7f7f7,stroke:#333,stroke-width:1px;
  class User,QQ,QCE,UI,SEC,OUT local;
```

这张图的关键依据是：导出格式与能力来自 QCE 主页与 Wiki；WebUI 端口、token 与 `security.json` 字段来自使用手册。[33]

关键模块（从文档可见性出发）
QCE 至少包含：会话列表（Sessions）、导出任务（Tasks）、导出格式生成（HTML/JSON/ZIP）、媒体下载器（图片/视频）、表情包导出器、计划任务/定时器。[34]

隐私合规风险与缓解
- 风险：任何“聊天记录导出工具”都可能触及群成员个人信息；下游若公开传播，很容易演化为“搬史/网暴/开盒”。（合规风险在 QQChatAnalyzer 的用户协议与 Chatlog 事件中也被反复强调。）[35]
- 缓解：你在“分析仪”侧应把 QCE 输出当作“用户自有数据”处理，并强制提示“需获得群成员明确同意”；对外导出默认脱敏、添加水印与审计日志（详见后文合规设计）。与 QCE 的 token/IP 白名单思路一致，你也应默认本地访问、最小暴露面。[36]

已知问题与改进建议（来自文档特征做工程判断）
- 多模式复杂度：Shell/Framework 两套运行形态会导致新手理解成本上升；“分析仪”侧更适合做“文件导入”为主、QCE API 对接为辅。[25]
- 安全性：token 存在本地文件，用户易误传；可在你的项目中提供“一键导入时自动读取 token”但要同时提示风险（napcat-qce-python 已实现自动令牌获取思路，可参考）。[37]

ChatLab 深入分析

定位与突出特性
ChatLab 是开源桌面应用，强调**本地优先（raw data 本地不强制上云）、流式解析 + 多 worker**支撑“大历史/百万消息”，并将不同平台导出映射为统一模型；同时集成 SQL 引擎与 AI Agents/Function Calling 用于搜索、总结与分析。[2]

官方给出的运行时架构（直接来自 README）
其 README 明确：
- Main Process（控制面）：Electron 主进程与 IPC、AI/i18n 服务；
- Worker Layer（计算面）：导入、索引、查询任务由 workerManager 执行；
- Renderer（交互面）：Vue3 + Nuxt UI + Tailwind；
- Data Pipeline：parser 自动识别格式 → 持久化（sessions/members/messages）→ 索引 → Query & Analysis → 可视化呈现。[2]

架构图（按 README 字段复原）

```
flowchart LR
  subgraph ElectronApp[ChatLab 桌面应用]
    R[Renderer\nVue3 + Nuxt UI + Tailwind]
    MP[Main Process\nlifecycle + IPC + AI/i18n]
    WK[Worker Layer\nimport/index/query workers]
    P[parser/\n格式识别与解析]
    DB[(本地存储\nsessions/members/messages)]
    IDX[(索引\nsession/time oriented)]
    Q[worker/query/*\n指标/关系/SQL Lab/AI探索]
  end

  R <--> MP
  MP <--> WK
  WK --> P --> DB --> IDX --> Q --> R

  classDef lab fill:#f7fbff,stroke:#2b6cb0,stroke-width:1px;
  class ElectronApp,R,MP,WK,P,DB,IDX,Q lab;
```

依据：ChatLab README 的“Runtime Architecture/Data Pipeline”字段。 [2]

部署方式
ChatLab 作为 Electron 桌面应用，本地开发要求 Node.js >= 20 与 pnpm，并提供 `pnpm dev/build` 等脚手架。 [2]

隐私合规与启示
ChatLab 的“Private by default / Local-first”原则对于你的项目非常关键：聊天记录分析天然涉及个人信息，默认本地存储与处理可以显著降低风险面。 [38]

已知问题与改进空间（面向你的项目）
- ChatLab 虽支持 QQ，但其统一模型是否覆盖“表情包/图片OCR/向量RAG”并未在 README 摘要里明确（需要读其标准化格式规范才可定论）。 [2]
- 你的项目若定位“QQ群搬史(屎)分析仪”，可以更激进地补齐“多模态 + RAG + 可控导出（脱敏水印）”这三块——这正是 ChatLab 生态中不明显的差异化。

QQChatAnalyzer 深入分析

定位
QQChatAnalyzer 是“QQ 文本聊天记录分析平台”，支持个人分析、群聊分析、社交网络可视化和 AI 摘要生成，但明确写明“本程序不支持图片分析”。 [12]

数据入口与解析方式
它支持两条导入路径：
- JSON：下载 QCE（qq-chat-exporter）导出 JSON 后导入；
- TXT：走 QQ 消息管理器导出 TXT，但作者提示该方法“会损失信息量（转发、表情、文件等数据）”。 [13]

其用户协议强调：工具不接入 QQ API、主要用正则匹配等技术对用户提供的特定格式文本进行提取分析；除 AI 总结外在本地运行；并要求用户确保已获得群聊成员明确同意。 [26]

架构与数据流（基于README导入说明复原）

```
flowchart TB
  IN1[QCE导出JSON] --> PARSE
  IN2[QQ导出TXT] --> PARSE

  PARSE[解析/清洗\n正则提取字段] --> STORE[(本地数据目录\n/texts + 缓存)]
  STORE --> ANA[分析模块\n个人/群聊/社交网络]
  ANA --> VIS[可视化/报表\nWeb页面]
  ANA -->|可选| LLM[AI摘要\n第三方API Key配置]

  classDef web fill:#fff7ed,stroke:#c2410c,stroke-width:1px;
  class IN1,IN2,PARSE,STORE,ANA,VIS,LLM web;
```

依据：其 README 中“导出方法（JSON/TXT）”“不支持图片分析”“AI 摘要与第三方服务商”描述。[39]

隐私合规风险与缓解
- 风险：若开启 AI，总结内容将传输给用户配置的第三方 AI 服务商；且用户需自行阅读服务商隐私政策。[26]
- 这为你的项目提供“合规产品需求”：默认离线；云端 LLM 必须显著提示、提供脱敏预处理、支持“仅发送摘要/仅发送检索片段”、并提供删除策略（后文详细给出）。

QQ-Chat-AI-Analyzer 深入分析

定位与边界
该项目自述为“依赖 qq-chat-exporter-master 搭建的本地部署轻量化QQ群聊消息分析工具”，用于生成年度报告等；并明确“仅做了对 qce 导出的 json 格式聊天记录的适配”。[14]

LLM 依赖与隐私提示
它在 README 中直接提醒：调用时可能把聊天记录完整上传至模型服务提供商，用户需衡量隐私与便利并自行承担责任。[14]

规模与性能约束
作者给出经验：过大聊天记录会导致分析效果不佳，推荐上传在 120MB 以下。[14]

架构与数据流（按其使用流程归纳）

```
flowchart TB
  QCEJSON[QCE导出JSON] --> UPLOAD[拖拽上传/导入]
  UPLOAD --> PRE[预处理/裁剪/采样(可选)]
  PRE --> LLM[调用外部LLM\n(用户配置API)]
  LLM --> HTML[生成HTML报告\n(可选二次美化)]
  HTML --> LOCAL[(本地缓存/历史记录)]
  LOCAL --> DL[下载/导出]

  classDef ai fill:#f0f9ff,stroke:#0369a1,stroke-width:1px;
  class QCEJSON,UPLOAD,PRE,LLM,HTML,LOCAL,DL ai;
```

依据：其 README 的“仅适配 qce json”“上传 120MB 建议”“会把聊天记录完整上传至服务商”“生成报告与下载流程”等描述。[14]

从该项目得到的工程启示
- 年度报告类产品容易做成“一次性生成”，但缺乏可复用的结构化存储、检索与增量更新能力；
- 你的项目若要长期可用，应把“报告生成”降级为“一个上层应用”，底座应是可扩展的消息仓库 + 索引 + RAG。
- 必须把“上传全量聊天记录给云端模型”改造成“最小必要上下文 + 可选本地模型”，与 PIPL 的“最小范围/最短保存期限/透明告知”等原则一致。 [40]

NapCatQQ 深入分析

定位
NapCat 自述为“基于 NTQQ 的现代协议端框架”，并尽量遵守 OneBot；其文档同时提醒 OneBot 协议“停滞”。 [41]

许可与对你项目的影响（极关键）
NapCat 的 LICENSE（raw 版本）显示为“Limited Redistribution License”，包含“未经主作者明确许可，禁止未授权使用/复制/修改/分发”，且“不得用于商业目的”，并限制修改版不得公开发布。 [42]
这意味着：**NapCat 并非通常意义的开源许可证（OSI Open Source）**，若你的项目目标是“开源”，不适合把 NapCat 作为可再分发依赖嵌入项目中；你最多只能把它作为“用户自行安装的外部组件”，并在文档中提示用户自行遵守其许可与当地法律。 [43]

《QQ群搬史(屎)分析仪》完整可执行项目方案

本节给出“从零到可用”的方案，默认偏好：中文、开源、非商业化（运营层面），预算/服务器资源/目标用户规模均**未指定**。

总体产品形态与总体架构

我建议把项目拆成三层：**导入层（ingestion）— 存储与索引层（data+index）— 应用层（analysis+RAG+UI）**，并采用“本地优先”作为默认部署（与 ChatLab 的原则一致）。 [2]

推荐总体架构（本地优先 + 可选服务化）

```
flowchart TB
  subgraph Sources[数据来源]
    S1[QCE导出\nJSON/ZIP/HTML + 媒体]
    S2[QQ消息管理器导出TXT]
    S3[官方QQ机器人\nWebhook实时消息(可选)]
  end

  subgraph Ingest[采集与解析层]
    P1[格式检测\n(JSON/ZIP/HTML/TXT)]
    P2[解析器插件\nQCE Parser / TXT Parser / Webhook Parser]
    P3[媒体处理\n图片/视频抽取\n表情包归档]
    P4[清洗与规范化\n统一消息模型]
  end

  subgraph Storage[存储与索引层]
    DB[(关系型存储\nSQLite/Postgres)]
    FTS[(全文索引\nSQLite FTS5 / ES可选)]
```

```
    VDB[(向量库\nChroma/Milvus/Weaviate/Pinecone)]
    OBJ[(对象存储\n本地目录/MinIO可选)]
  end

  subgraph AI[AI与分析层]
    OCR[PaddleOCR\n图片OCR]
    IMGEMB[图像嵌入\nCLIP/SigLIP等]
    TXTEMB[文本嵌入\nBGE-M3等]
    RERANK[重排序\nbge-reranker-v2-m3]
    RAG[RAG编排\n检索+上下文构建+引用溯源]
    LLM[LLM推理\n本地(vLLM/Ollama/llama.cpp)\n或云API(可选)]
  end

  subgraph App[应用层]
    UI[Web UI / 桌面UI]
    API[本地API服务\nFastAPI/Node]
    REP[报告生成\n日报/周报/年度报告]
    GOV[隐私与合规\n脱敏/同意/审计/加密]
  end

  Sources --> Ingest --> Storage --> AI --> App
  AI --> Storage
  App --> Storage

  classDef sys fill:#f7f7f7,stroke:#333,stroke-width:1px;
  class Sources,Ingest,Storage,AI,App sys;
```

设计依据与选型动机：
- QCE 提供“高质量历史导出 + 媒体下载 + 表情包导出”，适合作主数据来源。 24
- 官方 QQ 机器人提供 API/webhook，适合做“增量新消息”，且运营规范清晰规定采集用户数据应获同意、披露用途并按要求删除。 5
- PaddleOCR 为成熟开源 OCR 方案；CLIP 为经典图文共同嵌入方案；RAG 范式可参考原始论文。 44
- BGE-M3 支持 dense + sparse（token weights 类 BM25）有利于混合检索；bge reranker 推荐用于“先召回后重排”的检索增强。 45
- 本地 LLM 推理可选 vLLM（服务化高吞吐）、Ollama（易用）、llama.cpp（C/C++推理，GGUF 生态）。 46

推荐技术栈

在“未指定用户规模/资源”的前提下，我给出两套可执行栈：**本地桌面优先** 与 **自托管服务优先**。你可以先做 MVP（桌面/本地），再扩展到服务化。

本地桌面优先（默认推荐）
- 前端/桌面壳：Electron 或 Tauri（ChatLab 用 Electron 是可行案例）。 2
- UI：React 或 Vue3（ChatLab 使用 Vue3 + Nuxt UI + Tailwind）。 2
- 本地 API：Python FastAPI（便于接 ML）、或 Node.js（与 Electron 同栈）
- 本地存储：SQLite（含 FTS5），媒体文件走本地目录；后续可切 Postgres/MinIO
- 向量库：Chroma（本地简单），规模上来再接 Milvus/Weaviate。 47
- OCR：PaddleOCR（中文/复杂文档友好）。 48

- 多模态：CLIP（图像嵌入与相似检索）；可选 Qwen2.5-VL 做图像理解与生成式解释。[49]
- LLM：本地 Qwen2.5 系列（中文强、Apache-2.0）+ 推理框架 vLLM/Ollama/llama.cpp。[50]

自托管服务优先（团队/多人使用）
- 前端：Next.js / Vue
- 后端：FastAPI（REST）+ WebSocket（进度/流式RAG）
- Worker：Celery/RQ/自研队列（用于解析、OCR、embedding、报表）
- 向量库：Milvus/Weaviate（K8s/Compose）；或 Pinecone（托管）。[51]
- LLM Serving：vLLM（GPU 多并发）。[52]

QQ 接入与数据获取方案

这里必须区分“**历史聊天记录**”与“**实时新消息**”。

历史聊天记录（优先推荐：文件导入）
- QCE 支持导出 HTML/JSON/ZIP，并能完整下载图片/视频、导出表情包；非常适合作为“历史记录导出器”。[34]
- QQChatAnalyzer/annual-report-builder/QQ Summary Genius 等也都采用“QCE JSON 或 QQ 导出 TXT”路线，说明其工程可行性较高。[53]

实时新消息（可选：官方 QQ 机器人 webhook）
- 官方文档宣称“频道、群、单聊等场景全面打通”，并提供 API 与 webhook 事件。[21]
- 官方运营规范明确：收集用户个人信息前要获得同意、披露用途，用户要求应删除数据，终止使用应删除从 QQ 接收的用户数据等。[22]
- 现实约束：官方机器人通常只能拿到“机器人加入后产生的消息事件”，并不等价于“导出全部历史”；因此它更适合“持续增量更新、日报/周报”。

不建议作为你项目默认能力的方案：NapCat 直连/协议逆向
- NapCat 的许可对再分发/公开修改版等限制极强，并且包含非商业限制，与你“开源项目”的生态兼容性很差。[43]
- 即使技术上可行，协议类工具往往还伴随账号风控/平台条款风险（此处不展开具体绕过方法；建议以“文件导入 + 官方机器人”作为合规主路线）。

数据采集与解析细节

统一消息模型（Canonical Schema）
将不同来源映射到统一结构，关键字段建议包含：
- conversation：平台（QQ）、类型（群/私聊）、群号/群名（可脱敏）、时间范围
- participant：user_id（可哈希）、昵称（可脱敏）、角色（群主/管理员/成员，如果可得）
- message：message_id、timestamp、sender_id、text、elements（富文本元素数组）、reply_to、mentions、attachments、raw_source_ref（原始文件偏移/原始 id）
- attachment：type（image/video/file/sticker）、uri（本地路径）、sha256、pHash（可选）、width/height、mime、size
- derived：语言、token_count、主题、情绪、embedding_id、ocr_text、caption、safety_flags 等

表情包/图片处理管线
- “统计层”：图片/表情频次（QQLog 已有类似统计功能）。[17]
- “理解层”：
- OCR：用 PaddleOCR 对图片/表情包跑文字识别（适配中文、截图型表情包）。[48]
- 图像嵌入：用 CLIP 生成 embedding，用于“找相似表情包/找某个梗图”。[54]

- Caption/标签：可选 Qwen2.5-VL（3B/7B 更适合本地）为图片生成简短描述，便于纯文本检索。 [55]
- “存储层”：
- OCR 文本进 FTS；
- 图像 embedding 进向量库；
- caption 既可进 FTS 也可进向量库（做多路召回）。

Embeddings 与向量数据库选型对比

嵌入模型建议（中文优先）
- 文本嵌入：BGE-M3（支持 dense embedding + sparse token weights，可做混合检索）。 [56]
- 重排序：bge-reranker 系列（cross-encoder rerank，用于重排 top-k）。 [57]
- 图像嵌入：CLIP（通用图文对齐嵌入）。 [54]

向量数据库对比表（按你的要求：Milvus / Weaviate / Pinecone / Chroma）

维度	Milvus	Weaviate	Pinecone	Chroma
开源性	开源向量数据库，面向规模化检索；GitHub 描述强调高性能与可扩展。 [58]	开源，BSD-3-Clause。 [59]	商业托管为主（官网强调合规认证与托管能力）。 [60]	开源（docs 明确“open-source search engine for AI”）。 [61]
部署复杂度	中-高：分布式/云原生更强，适合更大规模；文档提供 Docker Compose/K8s 安装路径。 [62]	中：支持自托管（docker/k8s）；云版本另计。 [63]	低（用户视角）：托管服务；但需要处理网络/密钥/数据合规。 [60]	低：本地/嵌入式体验强，适合快速上手；docs 强调“everything built-in”。 [61]
性能与规模取向	强调“built for scale”、可处理大规模非结构化数据并高效检索。 [58]	强调“semantic search at scale”，并将向量检索、过滤、RAG 等整合在统一接口。 [64]	面向生产托管、SLA 与合规认证场景（官网列出 SOC2/GDPR/ISO27001/HIPAA 等）。 [60]	更偏本地与应用嵌入式检索，支持 dense/sparse/hybrid 与多模态检索场景。 [65]
成本（未指定预算下的说明）	软件本身开源；主要成本在自托管算力/存储/运维。 [66]	软件开源；自托管成本同上；若用云产品则按厂商计费。 [64]	按服务计费（具体价格与配额随版本变化）；另需评估数据出境/合规成本。 [60]	软件开源；本地成本最低，适合个人/小团队。 [65]
适合你项目的建议	规模不明时先别上：可作为“团队版/大群版”后续选项	若你偏好“把RAG能力集成在DB侧”，Weaviate 是候选	若你明确要托管与合规认证，且能接受商业依赖，可选	MVP 首选：落地快、依赖轻、可本地运行

RAG 实现细节

RAG 的基本思想是：在生成前先检索外部知识（你的聊天记录库），再把检索片段作为上下文输入生成模型，以提升事实性并降低幻觉——这与 RAG 原始论文的范式一致。 [67]

针对群聊的“RAG 文档切分”策略（强烈建议做成可配置）
- 按“会话窗口”切分：例如每 N 条消息或每 T 分钟一个 chunk（更贴近讨论上下文）。
- 按“时间断点”切分：超过 X 分钟无消息则新开 chunk（自然分段）。
- 按“主题聚类”切分：先 embeddings 后聚类（进阶）。

检索策略（推荐：Hybrid + Rerank）
- 初召回：
- dense：BGE-M3 向量召回；
- sparse：BM25/token weights（BGE-M3 支持生成 token weights 的思路可用于混合检索）。 [68]
- 重排：bge-reranker-v2-m3 对 top-k 进行 cross-encoder 相关性评分再排序（FlagEmbedding 与 bge-reranker 模型卡都给出“先召回再重排”的推荐用法）。 [69]
- 生成：只把“Top chunks + 必要的元信息（时间、人名脱敏标签、消息 ID）”喂给 LLM，并在回答中输出**引用溯源**（时间戳/消息链接/导出片段），避免“凭空编”。

推荐 LLM 与调用方式

本地模型（优先，隐私友好）
- 纯文本：Qwen2.5（0.5B-72B 多档）；模型卡明确其为最新系列，并给出参数范围。 [70]
- 多模态：Qwen2.5-VL 系列（3B/7B/72B 等），集合页显示其为“Vision-language model series based on Qwen2.5”。 [55]

推理/Serving 框架（按场景选择）
- vLLM：高吞吐、内存高效的 LLM serving 引擎，适合服务化部署与并发。 [52]
- Ollama：强调“快速运行开源模型”，适合本地/轻量部署。 [71]
- llama.cpp：C/C++ 推理，生态要求 GGUF 格式，适合 CPU/低资源/量化模型。 [72]

云模型（可选，默认关闭）
如果用户主动选择云 API，需要：
- 明示“会发送哪些内容”；
- 默认做脱敏与最小化；
- 提供“仅发送检索片段而非全量记录”的模式（避免重蹈 QQ-Chat-AI-Analyzer 的“可能上传全量聊天记录”风险提示）。 [73]

隐私与合规设计

这里给出“产品级”设计要点，核心目标是：**让工具更像 ChatLab/QQChatAnalyzer 那样“本地优先 + 用户自担授权义务”，并进一步提供工程化的保护措施**。 [74]

必须实现的机制（建议作为 MVP 里程碑的一部分）
- 明示同意与使用边界：首次导入必须确认“已获得群成员同意”“不用于公开传播/网暴”。（QQChatAnalyzer 用户协议已强调“必须确保已获得群聊成员明确同意”。） [26]
- 数据最小化：只导入用户选择的群与时间范围；符合 PIPL “最小范围/最短必要时间”的方向。 [75]
- 脱敏/去标识化：默认将 QQ 号、手机号、邮箱、身份证等做规则脱敏；昵称映射为可逆别名（本人可查看原值，分享版不可）。
- 加密：本地数据库加密（如 SQLCipher）+ 媒体文件可选加密；密钥托管 OS Keychain。
- 访问控制：本地口令/锁屏；多用户场景要 RBAC。
- 审计与防滥用：导出/分享动作记录审计日志；生成“分享版报告”时自动加水印（时间、导出者、脱敏级别）。
- 数据保留策略：提供“一键删除群数据/一键清空AI缓存”；与 PIPL“保存期限最短必要时间”一致。 [75]

- 若接入官方机器人：遵循其运营规范，确保采集前同意、披露用途、按用户要求删除、终止服务立即删除数据等。 22

性能与扩展性方案

你可以直接借鉴 ChatLab 的“Streaming over buffering”“Worker Layer”思想：对大体量聊天记录，必须用流式解析与多线程 worker，把 CPU 密集型任务（OCR、embedding、索引）从 UI 线程剥离。 2

建议的工程策略：
- 导入阶段：分批写入（batch insert）、可中断/可续跑；
- 媒体处理：异步队列，优先抽取元数据（尺寸/哈希）再做 OCR；
- 向量化：对历史进行离线批处理，对新增做增量；
- 查询阶段：FTS + Vector 双路召回，避免所有查询都走 LLM；
- 报告生成：可缓存、可复用中间统计结果。

测试与评估指标

建议把“可测性”写进 MVP：
- 解析正确性：导入消息总数一致性、时间戳解析准确率、元素类型覆盖率（文本/图片/表情/转发等）。
- 检索质量：Recall@k、MRR（用你自建的“问答-证据”小集评测）。
- OCR 质量：字符准确率/字段抽取准确率（可用脱敏样本集）。
- RAG 回答质量：引用覆盖率（回答中是否给出处）、事实一致性（人工抽检）。
- 性能：导入吞吐（msg/s）、查询延迟（p50/p95）、向量化耗时、磁盘占用。

开源发布与社区治理建议

- 许可证选择：若你强调“改动必须回馈”，可选 AGPL（ChatLab 使用 AGPL）。 2 若你想扩大复用，可选 MIT/Apache-2.0（annual-report-builder、QQChatAnalyzer、QQ Summary Genius 都有 MIT 案例）。 76
- 需要特别提醒：**“禁止商用”通常不被 OSI 认为是开源**（NapCat 的许可就是这种“source-available/限制性”范式）。 42 若你坚持“非商业化许可”，需接受“可能不被开源社区认可、生态协作受阻”的代价。
- 安全披露：设置 SECURITY.md、漏洞提交流程；聊天数据工具属于高敏项目。
- 数据样本：提供“脱敏生成器”自动造测试数据，避免仓库出现真实聊天记录。
- 反滥用条款：在 README 与 UI 中明确禁止网暴/开盒用途，并提供“分享版默认脱敏+水印”作为技术约束（条款不足以阻止滥用，但能显著降低误用概率）。

开发路线图与里程碑

以下给出按“周”为单位的 12 周路线图（可缩放）。人力估算以“未指定资源”为前提，提供两档：
- **1人全栈（偏慢）**：12~16 周做出可用 MVP；
- **2人（后端/数据+前端/产品）**：10~12 周做出可用 MVP，并开始多模态与RAG。

MVP 功能清单

MVP 的定义：可导入、可查、可统计、可生成“安全分享版摘要”，且默认不出网。
- 导入：QCE JSON（必选），QQ TXT（可选） 77
- 存储：SQLite（消息、成员、会话、附件元信息）
- 检索：FTS（关键词/按人/按时间过滤）
- 基础分析：活跃度、关键词、时段热力、@关系简图（不必一开始做复杂图谱）

- 报告：一键生成“周报/年度梗概（简版）”，默认脱敏
- 可选AI：本地小模型总结（或用户自配 API，但必须脱敏预览与二次确认） [78]

12 周里程碑表

周期	目标	交付物	风险控制点
第1-2周	项目脚手架与统一数据模型	repo 初始化、CI、SQLite schema v0、导入插件接口	写清“仅处理用户自主导出数据”；默认本地；隐私告知弹窗
第3-4周	QCE JSON 导入 + 基础统计	QCE Parser、消息/成员/会话落库、基础统计页面	导入过程不上传；日志中不打印敏感字段
第5-6周	检索与导出	SQLite FTS 搜索；导出“脱敏分享版 HTML/MD”	导出默认脱敏+水印；导出审计日志
第7-8周	多模态最小闭环	媒体文件归档、图片元数据、PaddleOCR 接入（可异步）	OCR 默认本地；缓存可清除；资源限额
第9-10周	Embeddings + 向量库（本地）	Chroma 接入；文本 embedding（BGE-M3）；混合检索雏形	embedding 任务队列化；可关闭；加密存储可选
第11周	RAG 问答 Beta	“问群里讨论过什么/谁提到过X”RAG；引用消息ID与时间	强制引用溯源；限制输出敏感字段；提示“非法律建议”等
第12周	开源发布准备	文档、贡献指南、示例数据生成器、Roadmap、版本 v0.1	许可证确定；安全披露流程；反滥用声明

示例 API 与数据库 Schema

下面给出一套“可直接落地”的最小 API 与 schema（面向本地服务）。字段可按实际来源扩展。

示例 API 设计（REST）

方法	路径	用途	请求要点	响应要点
POST	`/api/imports`	创建导入任务	`{source_type:"qce_json", file_path:"..."}`	`{import_id, status}`
GET	`/api/imports/{id}`	查询导入进度	-	`{status, processed_messages, errors}`
GET	`/api/conversations`	列出会话/群	filter: platform/type/time_range	会话列表（脱敏字段可切换）
GET	`/api/messages`	拉取消息	`conversation_id`, `from`, `to`, `sender`	消息列表（分页）
POST	`/api/search`	检索（FTS/向量混合）	`{query, filters, mode:"fts|vector|hybrid"}`	命中片段 + 证据引用

方法	路径	用途	请求要点	响应要点
POST	`/api/rag/answer`	RAG 问答	`{question, filters, llm_profile}`	`{answer, citations: [{message_id, ts}]}`
POST	`/api/reports`	生成报告	`{conversation_id, range, template, redaction_level}`	`{report_id, download_url}`
GET	`/api/reports/{id}`	获取报告/元信息	-	报告内容或下载链接

示例数据库 Schema（核心表）

表	主键	关键字段	说明
`conversations`	`conversation_id`	`platform, type, title, external_id_hash`	群/私聊元信息（external_id 建议哈希化存储）
`participants`	`participant_id`	`conversation_id, user_id_hash, display_name_redacted`	群成员（user_id 建议哈希；原值可选加密表保存）
`messages`	`message_id`	`conversation_id, ts, sender_id, text, elements_json, reply_to`	统一消息表：text+富文本 elements
`attachments`	`attachment_id`	`message_id, type, path, sha256, phash, mime, size`	图片/视频/表情包等
`ocr_results`	`attachment_id`	`text, blocks_json, engine, created_at`	PaddleOCR 输出（可重跑） [79]
`embeddings`	`embedding_id`	`owner_type, owner_id, model, vector_ref, created_at`	向量索引引用（向量本体进 Chroma/Milvus 等） [80]
`consents`	`consent_id`	`conversation_id, scope, evidence, created_at`	记录“已获同意”的管理数据（合规审计）
`audit_logs`	`audit_id`	`action, actor, target, ts, meta_json`	导出/分享/删除等敏感操作审计

可能的法律与伦理风险与缓解建议

> 说明：以下为工程与合规风险梳理，不构成正式法律意见；如要面向公众发布或涉及组织/企业使用，建议咨询专业律师或合规团队。

个人信息保护与数据合规风险

风险点

- 群聊内容往往包含可识别个人的信息（账号、昵称、联系方式、聊天内容、图片等），属于《个人信息保护法》所称“与已识别或可识别自然人有关的信息”；处理活动包括收集、存储、使用、加工、传输、提供、公开等。 [75]

- PIPL 强调处理个人信息应遵循合法、正当、必要、最小范围、公开透明与安全保障等原则，并要求在很多场景下取得同意、提供撤回同意与删除机制。 75
- 一旦用户将分析结果公开分享，可能构成对群成员隐私权的侵害，甚至引发现实伤害。

缓解建议（工程化）
- 默认本地处理、默认不出网（ChatLab 的“Private by default”方向）。 2
- 默认脱敏输出、分享版强制水印、导出审计日志。
- 提供“删除某成员/某群数据”的功能与数据保留策略（响应 PIPL 的删除与最短保存期限要求）。 75
- 对云端 LLM：采取“最小必要片段上传、先脱敏预览、用户二次确认”，避免全量上送（QQ-Chat-AI-Analyzer 已提示全量上传隐患）。 73

平台规则与账号风险

- 若你采用官方 QQ 机器人接入，必须遵守其运营规范，其中明确要求收集用户数据前要经过用户同意、披露用途、按要求删除，并限制将数据用于未授权用途。 22
- 若你采用非官方协议端工具（如 NapCat 生态），除平台条款风险外，NapCat 本身许可也对使用/分发有严格限制，可能与你“开源项目”目标冲突。 43

滥用与网暴伦理风险

项目名称“搬史(屎)”本身带有“翻旧账/曝光”的语境，极容易被用于：
- **断章取义传播、挂人、网暴**
- **“开盒”关联信息**（从聊天记录中拼接身份线索）
- 诱导用户上传他人聊天记录并公开传播

缓解建议：
- 产品层面：默认脱敏、默认本地、默认“私密分析”，把“可公开分享”变成高摩擦流程（多次确认+水印+审计）。
- 治理层面：仓库 README 明确禁止用于网暴/开盒；对 Issue/PR 中出现的“采集他人数据/绕过平台限制”需求一律拒绝合并。
- 教育层面：在 UI 提示“你必须获得群成员同意”“请勿公开传播未经同意的聊天内容”（QQChatAnalyzer 也要求“必须确保已获得群成员明确同意”）。 26

参考资源与优先检索来源

为便于你后续持续调研与动手实现，这里按“优先级”给出推荐检索源（尽量中文/官方优先，必要时引用英文原始资料）。

QQ 数据获取/生态（优先）
- QCE / qq-chat-exporter：导出 HTML/JSON/ZIP、下载媒体、表情包导出、两种运行模式与本地 token 机制。 81
- QQ 官方机器人文档与运营规范：API/webhook 事件与“采集数据需同意/按请求删除”等规则。 5

聊天分析应用（对标学习）
- ChatLab：本地优先、流式解析、多 worker、SQL+AI Agent 的架构与数据管线描述。 2
- QQChatAnalyzer：导入路径（QCE JSON/TXT）、不支持图片分析、AI 摘要外发风险提示与用户协议写法。 39
- QQ-Chat-AI-Analyzer / QQ Summary Genius：LLM 报告生成与“文件体积/上传风险”经验。 82
- chatlog 移除事件：聊天记录工具潜在合规风险的真实案例警示。 20

RAG/检索与向量数据库（原理与工程）
- RAG 原始论文（arXiv/NeurIPS）：检索增强生成范式。 [67]
- Milvus 官方文档（中文）与 GitHub：部署与规模化向量检索能力。 [62]
- Weaviate GitHub（开源、BSD-3-Clause，含 RAG/重排等介绍）。 [63]
- Chroma 官方 docs：开源、支持 dense/sparse/hybrid 与多模态检索。 [61]
- Pinecone 官网：托管向量数据库、合规认证声明（用于评估云方案）。 [60]

Embedding / Rerank（中文可用）
- BGE-M3（dense + sparse token weights），适合混合检索。 [68]
- bge-reranker（推荐“先召回后重排”）。 [69]
- CLIP 原论文：图文对齐嵌入基础。 [54]

OCR（中文优先）
- PaddleOCR GitHub 与文档：开源 OCR 与文档解析能力。 [48]

本地推理与部署
- vLLM：高吞吐 LLM serving。 [52]
- Ollama：便捷运行开源模型。 [71]
- llama.cpp：GGUF 生态、C/C++ 推理。 [72]
- Qwen2.5 / Qwen2.5-VL：中文基础模型与多模态系列（模型卡/集合页）。 [83]

法律法规（官方优先）
- 《中华人民共和国个人信息保护法》（中国人大网原文）。 [75]
- 《中华人民共和国数据安全法》《中华人民共和国网络安全法》（中国人大网原文）。 [84]
- 两高《办理侵犯公民个人信息刑事案件适用法律若干问题的解释》（最高法公报/最高检网站均可查）。 [85]

---

[1] [4] [9] [23] [24] [25] [31] [32] [36] [81] https://github.com/shuakami/qq-chat-exporter/wiki/%E4%BD%BF%E7%94%A8%E6%89%8B%E5%86%8C
https://github.com/shuakami/qq-chat-exporter/wiki/%E4%BD%BF%E7%94%A8%E6%89%8B%E5%86%8C

[2] [30] [38] [74] https://github.com/hellodigua/ChatLab
https://github.com/hellodigua/ChatLab

[3] [12] [13] [26] [35] [39] [53] [77] [78] https://github.com/CutrelyAlex/QQChatAnalyzer
https://github.com/CutrelyAlex/QQChatAnalyzer

[5] [21] https://bot.q.qq.com/wiki/
https://bot.q.qq.com/wiki/

[6] [44] [48] [79] https://github.com/PaddlePaddle/PaddleOCR
https://github.com/PaddlePaddle/PaddleOCR

[7] [47] [61] [65] [80] https://docs.trychroma.com/
https://docs.trychroma.com/

[8] [40] [75] https://www.npc.gov.cn/npc/c2/c30834/202108/t20210820_313088.html
https://www.npc.gov.cn/npc/c2/c30834/202108/t20210820_313088.html

[10] https://github.com/shuakami/qq-chat-exporter
https://github.com/shuakami/qq-chat-exporter

[11] [37] https://github.com/streetartist/napcat-qce-python
https://github.com/streetartist/napcat-qce-python

14 73 82 https://github.com/JUSTMONIKA2022/QQ-Chat-AI-Analyzer
https://github.com/JUSTMONIKA2022/QQ-Chat-AI-Analyzer

15 16 76 https://github.com/ping-xiong/annual-report-builder
https://github.com/ping-xiong/annual-report-builder

17 https://github.com/usamimeri/QQLog
https://github.com/usamimeri/QQLog

18 29 https://github.com/JyiDeng/QQ-Summary-Genius/blob/master/README.md
https://github.com/JyiDeng/QQ-Summary-Genius/blob/master/README.md

19 https://github.com/JyiDeng/QQ-Summary-Genius
https://github.com/JyiDeng/QQ-Summary-Genius

20 https://github.com/sjzar/chatlog
https://github.com/sjzar/chatlog

22 https://bot.q.qq.com/wiki/business/
https://bot.q.qq.com/wiki/business/

27 28 https://github.com/ping-xiong/annual-report-builder/blob/main/README.md
https://github.com/ping-xiong/annual-report-builder/blob/main/README.md

33 34 https://qce.sdjz.wiki/
https://qce.sdjz.wiki/

41 https://github.com/NapNeko/NapCatQQ
https://github.com/NapNeko/NapCatQQ

42 43 https://raw.githubusercontent.com/NapNeko/NapCatQQ/main/LICENSE
https://raw.githubusercontent.com/NapNeko/NapCatQQ/main/LICENSE

45 56 68 https://huggingface.co/BAAI/bge-m3
https://huggingface.co/BAAI/bge-m3

46 52 https://github.com/vllm-project/vllm
https://github.com/vllm-project/vllm

49 54 https://arxiv.org/abs/2103.00020
https://arxiv.org/abs/2103.00020

50 70 83 https://huggingface.co/Qwen/Qwen2.5-7B
https://huggingface.co/Qwen/Qwen2.5-7B

51 62 66 https://milvus.io/docs/zh
https://milvus.io/docs/zh

55 https://huggingface.co/collections/Qwen/qwen25-vl
https://huggingface.co/collections/Qwen/qwen25-vl

57 69 https://huggingface.co/BAAI/bge-reranker-base
https://huggingface.co/BAAI/bge-reranker-base

58 https://github.com/milvus-io/milvus
https://github.com/milvus-io/milvus

59 https://github.com/weaviate/weaviate/blob/master/LICENSE
https://github.com/weaviate/weaviate/blob/master/LICENSE

60 https://www.pinecone.io/
https://www.pinecone.io/

63 64 https://github.com/weaviate/weaviate
https://github.com/weaviate/weaviate

67 https://arxiv.org/abs/2005.11401
https://arxiv.org/abs/2005.11401

71 https://github.com/ollama/ollama
https://github.com/ollama/ollama

72 https://github.com/ggml-org/llama.cpp
https://github.com/ggml-org/llama.cpp

84 https://www.npc.gov.cn/npc/c2/c30834/202106/t20210610_311888.html
https://www.npc.gov.cn/npc/c2/c30834/202106/t20210610_311888.html

85 https://gongbao.court.gov.cn/Details/f9bbdd79e39de33c7b3417fa4061ff.html?sw=%E4%BE%B5%E7%8A%AF
https://gongbao.court.gov.cn/Details/f9bbdd79e39de33c7b3417fa4061ff.html?sw=%E4%BE%B5%E7%8A%AF